

Prison Break

Exam Task — Java OOP

Overview

The goal of this task is to create a simplified text game simulating a prisoner's escape from a prison. The prison is represented by a 5x5 grid. Some cells contain **bushes** — dense vegetation that blocks the guards' line of sight and prevents them from passing through, but does NOT block the prisoner (he can hide there). Bushes appear on every other row in the first and last column. The simulation runs in discrete steps controlled by player input.

Prisoner

The prisoner is always on one cell of the grid. He starts at position (0, 0). Each turn he moves in the direction entered by the player, if the prison boundary allows it. He can move into bush cells (bushes are not an obstacle for him). If the move would go outside the grid, he stays in place. The player can also enter STOJ (STAY) — the prisoner does not move but guards still act.

Bushes

Bushes are a static obstacle that blocks guards' line of sight. Guards cannot see through or past bushes, and cannot move into bush cells. The prisoner CAN move through bushes — they provide cover from guards with wide-angle vision.

Guards — General

After the prisoner moves, all guards update. Each guard first performs its action, then checks if it can see the prisoner. If at least one guard spots the prisoner, the game ends with capture.

Guard Types

Hliadka (Patrol) — moves horizontally (left or right). Each turn it moves one cell in its current direction. If it hits a wall (grid edge) or a bush, it reverses direction and does NOT move further that turn. It can see the prisoner if he is in the direction it is facing, within **at most 3 cells**. Vision is blocked by bushes — neither into nor past a bush cell.

StrazcaVoVezi (Tower Guard) — does not move. It sleeps for 5 turns, then looks for 1 turn, repeating indefinitely. It can see everywhere EXCEPT bush cells — it captures the prisoner on any cell that is NOT a bush.

Input Format

During the game the player enters commands via standard input, one per line:

Command	Description
HORE	Move prisoner one cell up
DOLE	Move prisoner one cell down
VLAVO	Move prisoner one cell left
VPRAVO	Move prisoner one cell right
STOJ	Prisoner stays, guards still act
KONIEC	End game without result

Output Format

The **vypis()** method prints the full 5x5 grid. Cell display priority (first matching rule applies):

- 'u' — prisoner is on this cell
- '#' — bush cell
- guard representation character — if a guard is on this cell (guards never stand in bushes)
- '.' — empty cell

Coordinates start at top-left (0,0). Example: prisoner at (0,0), patrols at (1,1) facing right and (2,2) facing left, tower guard at (3,2) with 5 turns until it looks:

```
u . . . .
#> . . #
. . < . .
# . 5 . #
. . . . .
```

When a guard catches the prisoner, print:

```
Utecenec boľ chytený!
```

Class Descriptions

Pozicia (8 pts) — immutable grid coordinates (row, col). Grid size is ROZMER = 5.

- ROZMER: final int = 5
- new(row: int, col: int)
- jeRovnaka(other: Pozicia): boolean — true if positions are equal
- jePlatna(rozmer: int): boolean — true if coords are in range [0, ROZMER)
- jeKrovie(): boolean — true if this position contains a bush (every other row, first and last column)
- posun(dir: Smer): Pozicia — returns new position shifted by 1 in given direction

Smer (2 pts) — enum for movement direction: HORE, DOLE, VLAVO, VPRAVO.

- getOpacny(): Smer — returns the opposite direction (UP<=>DOWN, LEFT<=>RIGHT)

Strazca (4 pts) — interface for all guards.

- aktualizuj(prisoner: Pozicia): boolean
- getReprezentacia(): char
- jeNa(pos: Pozicia): boolean

Hliadka (11 pts) — horizontal patrol guard.

- new(pos: Pozicia, dir: Smer)
- aktualizuj (7 pts) — moves in current direction. If it would hit a wall or bush, reverses direction and stops. Then checks if prisoner is visible (same direction, max 3 cells, bushes block vision).
- getReprezentacia(): '>' if moving right, '<' if moving left
- jeNa(pos): true if guard is at that position

StrazcaVoVezi (7 pts) — tower guard with wide-angle vision.

- new(pos: Pozicia) — creates guard that will sleep for 5 more turns
- aktualizuj (3 pts) — updates state; if currently looking, checks if prisoner is on any non-bush cell
- getReprezentacia(): 'V' if looking, otherwise number of turns remaining until it looks again
- jeNa(pos): true if guard is at that position

Vaznica (13 pts) — manages the entire game board.

- new (2 pts) — prisoner starts at (0,0); Hliadka at (1,1) facing right; Hliadka at (2,2) facing left; StrazcaVoVezi at (3,2)
- posunUtecenca(dir: Smer) (2 pts) — moves prisoner if valid, otherwise stays
- aktualizujStrazov()(3 pts) — updates all guards; returns true if any guard catches prisoner
- vypis() (4 pts) — prints current state according to output format rules
- getStrazana(pos: Pozicia) (2 pts) — returns Optional<Strazca> at position, or empty

Main (4 pts) — *creates Vaznica instance; loops until player types KONIEC or prisoner is caught:*

1. Print current state
2. Read player command
3. Process command
4. Check end condition

After guard update, if prisoner is caught, print the final map state.

Important Rules

- OOP principles must be followed throughout

- Violating information hiding = **0 points**
- Methods with syntax errors will not be graded

Allowed Resources

- Javadoc: <https://docs.oracle.com/en/java/javase/21/docs/api/index.html>
- Google Translate (on computer)
- Windows calculator or non-programmable calculator
- Paper and pen